

1. Write a Lex/YACC program to remove all #define macros from a given C program.

**input.c file:**

```
#define MAX 10
#define LOW 0
#include<stdio.h>
#include<math.h>
int main()
{
    printf("\nHello World!");
    return 0;
}
```

**Program:**

```
%{
#include <stdio.h>
%}

%%

^[ \t]*#[ \t]*define[^\n]*\n    ;

.|\\n    fprintf(yyout, "%s", yytext);

%%

int main()
{
    yyin = fopen("input.c", "r");
    yyout = fopen("output.c", "w");

    yylex();

    fclose(yyin);
    fclose(yyout);

    printf("Macros removed successfully.");
    return 0;
}

int yywrap()
{
    return 1;
}
```

### Output:

```
D:\CompilerLab>flex "1. Macros.l"
D:\CompilerLab>gcc lex.yy.c
D:\CompilerLab>a.exe input.c
Macros removed successfully.
```

### Ouput.c file:

```
#include<stdio.h>
#include<math.h>
int main()
{
    printf("\nHello World!");
    return 0;
}
```

2. Write a Lex/YACC program to count the number of characters, words, sentences, and lines in a given paragraph.

### **Program:**

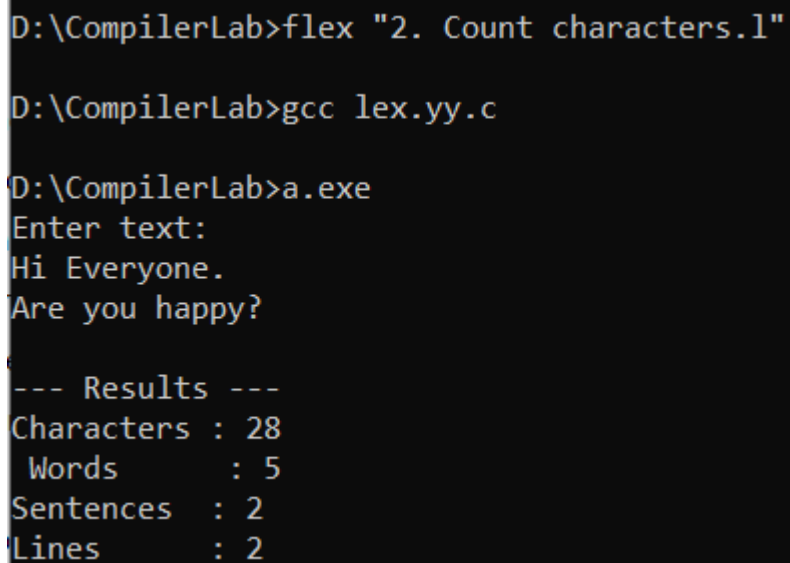
```
%{
#include <stdio.h>
int char_count = 0;
int word_count = 0;
int sentence_count = 0;
int line_count = 0;
}%

%%
\n          { char_count++; line_count++; }
[.!?]       { char_count++; sentence_count++; }
[a-zA-Z0-9]+ { char_count += yyleng; word_count++; }
.           { char_count++; }
%%

int main() {
    printf("Enter text:\n");
    yylex();
    printf("\n--- Results ---\n");
    printf("Characters : %d\n Words      : %d\n Sentences : %d\n Lines      : %d\n", char_count, word_count, sentence_count, line_count);
    return 0;
}

int yywrap() {
    return 1;
}
```

### **Output:**



```
D:\CompilerLab>flex "2. Count characters.l"

D:\CompilerLab>gcc lex.yy.c

D:\CompilerLab>a.exe
Enter text:
Hi Everyone.
Are you happy?

--- Results ---
Characters : 28
Words      : 5
Sentences  : 2
Lines      : 2
```

3. Write a Lex/YACC program to count the number of digits and special characters (&, #, \_, -, \$ etc.) in a given sentence.

### **Program:**

```
%{
#include<stdio.h>
int count_digits=0, special_char=0;
%}

%%
[0-9] {count_digits++;}
[\\&\\#\\_\\-\\$] {special_char++;}
\\n {printf("\\nNumber of digits=%d\\nNumber of special characters=%d",count_digits,special_char);}

%%
int main()
{
printf("\\nEnter a sentence:");
yylex();
return 0;
}
int yywrap()
{
return 1;
}
```

### **Output:**

```
D:\CompilerLab>flex "3. Count digits.l"
D:\CompilerLab>gcc lex.yy.c
D:\CompilerLab>a.exe

Enter a sentence:Hi123456 and Bye#$_- $
Hi and Bye
Number of digits=6
Number of special characters=5
```

4. Write a Lex/YACC program to check whether a given sentence is declarative or interrogative.

**Program:**

```
%{
    #include<stdio.h>

}%

%%
.*\? {printf("\nInterrogative Sentence.\n");}
.*\. {printf("\nDeclarative Sentence.\n");}
%%
int main()
{
    printf("\nEnter a sentence:\n");
    yylex();
    return 0;
}

int yywrap()
{
    return 1;
}
```

**Output:**

```
D:\CompilerLab>flex "4. Check Sentence.l"
D:\CompilerLab>gcc lex.yy.c
D:\CompilerLab>a.exe

Enter a sentence:
Who are you?

Interrogative Sentence.

I am a MCA student.

Declarative Sentence.
```

5. Write a Lex/YACC program to recognize and count the identifiers, and integer constants from a given input text.

### **Program:**

```
%{
#include <stdio.h>

int id_count = 0;
int int_count = 0;
%}

%%

[a-zA-Z][a-zA-Z0-9_]* {
    printf("Identifier: %s\n", yytext);
    id_count++;
}

[0-9]+ {
    printf("Integer Constant: %s\n", yytext);
    int_count++;
}

[ \t\n]+ ;

. ;

%%

int yywrap()
{
    return 1;
}

int main()
{
    printf("Enter the input text:\n");
    yylex();

    printf("\nNumber of Identifiers = %d\n", id_count);
    printf("Number of Integer Constants = %d\n", int_count);

    return 0;
}
```

### **Output:**

```
D:\CompilerLab>flex "5. Count Identifier.l"
D:\CompilerLab>gcc lex.yy.c
D:\CompilerLab>a.exe
Enter the input text:
int sum=90+10;
Identifier: int
Identifier: sum
Integer Constant: 90
Integer Constant: 10

Number of Identifiers = 2
Number of Integer Constants = 2
```

6. Write a Lex/YACC program to count the number of blank lines from a given text.

**Program:**

```
%{
#include <stdio.h>

int blank_lines = 0;
%}

%%

^[ \t]*\n      { blank_lines++; }

.*\n           ;

%%

int yywrap()
{
    return 1;
}

int main()
{
    printf("Enter the text:\n");
    yylex();

    printf("\nNumber of blank lines = %d\n", blank_lines);

    return 0;
}
```

**Output:**

```
D:\CompilerLab>flex "6. count blank lines.l"
D:\CompilerLab>gcc lex.yy.c
D:\CompilerLab>a.exe
Enter the text:
Vidyasagar University

I am a MCA student.

Hi.

Number of blank lines = 4
```

7. Write a Lex/YACC program to implement a calculator that evaluates basic arithmetic expressions.

**Program:**

**calc.l**

```
%{
#include "calc.tab.h"
#include <stdlib.h>
}%

%%

[0-9]+      { yylval = atoi(yytext); return NUMBER; }
[ \t]       ;
\n          { return '\n'; }
.           { return yytext[0]; }

%%

int yywrap(void)
{
    return 1;
}
```

**calc.y**

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
void yyerror(const char *s);
}%

%token NUMBER

%left '+' '-'
%left '*' '/'

%%

input:
    expr '\n'      { printf("Result = %d\n", $1); }
    ;
```

---

```

expr:
| expr '+' expr { $$ = $1 + $3; }
| expr '-' expr { $$ = $1 - $3; }
| expr '*' expr { $$ = $1 * $3; }
| expr '/' expr {
    if ($3 == 0)
    {
        yyerror("Division by zero");
        YYABORT;
    }
    $$ = $1 / $3;
}
| '(' expr ')' { $$ = $2; }
| NUMBER      { $$ = $1; }
;

%%

void yyerror(const char *s)
{
    printf("Error: %s\n", s);
}

int main(void)
{
    printf("Enter expression: ");
    yyparse();
    return 0;
}

```

### Output:

```

(kali@kali)-[~/Desktop/Compiler]
$ bison -d calc.y

```

```

(kali@kali)-[~/Desktop/Compiler]
$ flex calc.l

```

```

(kali@kali)-[~/Desktop/Compiler]
$ gcc calc.tab.c lex.yy.c -o calculator

```

```

(kali@kali)-[~/Desktop/Compiler]
$ ./calculator

```

```

Enter expression: 8+9
Result = 17

```

8. Write a Lex/YACC program to convert a given binary number to its decimal equivalent.

**Program:**

```
%{
#include <stdio.h>

int decimal = 0;
%}

%%

[01]+ {
    decimal = 0;

    for (int i = 0; yytext[i] != '\0'; i++)
    {
        decimal = decimal * 2 + (yytext[i] - '0');
    }

    printf("Decimal Equivalent = %d\n", decimal);
}

[ \t\n]+ ;

. {
    printf("Invalid Binary Number\n");
}

%%

int yywrap()
{
    return 1;
}

int main()
{
    printf("Enter a binary number: ");
    yylex();

    return 0;
}
```

### Output:

```
D:\CompilerLab>flex "8. binary to decimal.1"
```

```
D:\CompilerLab>gcc lex.yy.c
```

```
D:\CompilerLab>a.exe
```

```
Enter a binary number: 1010
```

```
Decimal Equivalent = 10
```

```
D:\CompilerLab>a.exe
```

```
Enter a binary number: 10111
```

```
Decimal Equivalent = 23
```

9. Write a Lex/YACC program to check whether the parentheses in a given expression are well-formed or not.

**Program:**

```
%{
#include <stdio.h>

int count = 0;
int valid = 1;
%}

%%

"("      { count++; }

")"      {
            count--;

            if (count < 0)
            {
                valid = 0;
            }
        }

\n       {
            if (count == 0 && valid == 1)
                printf("Parentheses are Well-Formed\n");
            else
                printf("Parentheses are Not Well-Formed\n");

            return 0;
        }

.        ;

%%

int yywrap()
{
    return 1;
}

int main()
{
    printf("Enter an expression: ");
    yylex();

    return 0;
}
```

### Output:

```
D:\CompilerLab>flex "9. Well formed.l"
```

```
D:\CompilerLab>gcc lex.yy.c
```

```
D:\CompilerLab>a.exe
```

```
Enter an expression: (A+B)-C
```

```
Parentheses are Well-Formed
```

```
D:\CompilerLab>a.exe
```

```
Enter an expression: )(A+B))
```

```
Parentheses are Not Well-Formed
```

10. Write a Lex/YACC program to prepend line numbers to each line in a given file.

**temp.txt file**

```
Hi Everyone.  
Vidyasagar University.  
MCA Student.
```

**Program:**

```
%{  
int yylineno;  
%}  
  
%%  
  
^(.*)\n printf("%4d\t%s",++yylineno,yytext);  
%%  
  
int main(int argc, char *argv[])  
{  
    yyin = fopen(argv[1], "r");  
    yylex();  
    fclose(yyin);  
    return 0;  
}  
  
int yywrap()  
{  
    return 1;  
}
```

**Output:**

```
D:\CompilerLab>flex "10. Prepend Line.1"  
  
D:\CompilerLab>gcc lex.yy.c  
  
D:\CompilerLab>a.exe temp.txt  
1    Hi Everyone.  
2    Vidyasagar University.  
3    MCA Student.
```